

EC4219: Software Engineering (Spring 2024)

Mid-term Exam

Open-book Test

Date and Time: 4/15, 13:00 – 14:15

Place: EECS Building B203

Instructor: Sunbeom So

Student ID: _____

Name: _____

* Leave the score table blank

	Score Range	Your Score
Problem 1	0 – 20	
Problem 2	0 – 20	
Problem 3	-15 – 15	
Problem 4	0 – 15	
Problem 5	-30 – 30	
Total	0 – 100	

Problem 1. (20pt) Propositional Logic

Consider the formula F

$$F : ((P \rightarrow Q) \wedge (Q \rightarrow R) \wedge (R \rightarrow S)) \rightarrow (P \rightarrow S).$$

Below are the available deduction rules for the semantic argument method.

$\frac{I \models \neg F}{I \not\models F} (R1)$	$\frac{I \not\models \neg F}{I \models F} (R2)$	$\frac{I \models F \wedge G}{I \models F, I \models G} (R3)$	$\frac{I \not\models F \wedge G}{I \not\models F \mid I \not\models G} (R4)$
$\frac{I \models F \vee G}{I \models F \mid I \models G} (R5)$	$\frac{I \not\models F \vee G}{I \not\models F, I \not\models G} (R6)$	$\frac{I \models F \rightarrow G}{I \not\models F \mid I \models G} (R7)$	$\frac{I \not\models F \rightarrow G}{I \models F, I \not\models G} (R8)$
$\frac{I \models F \leftrightarrow G}{I \models F \wedge G \mid I \models \neg F \wedge \neg G} (R9)$	$\frac{I \not\models F \leftrightarrow G}{I \models F \wedge \neg G \mid I \models \neg F \wedge G} (R10)$		
$\frac{I \models F \quad I \models F \rightarrow G}{I \models G} (R11)$	$\frac{I \models F \quad I \not\models F}{I \models \perp} (R12)$		

Complete the proof to demonstrate that F is valid (2pt for each).

S1.	$I \not\models F$	assumption
S2.	$I \models (P \rightarrow Q) \wedge (Q \rightarrow R) \wedge (R \rightarrow S)$	by S1 and R8
S3.	$I \not\models (P \rightarrow S)$	by S1 and R8
S4.	$I \models P$	by S3 and R8
S5.	$I \not\models S$	by S3 and R8
S6.	$I \models P \rightarrow Q$	_____
S7.	$I \models (Q \rightarrow R) \wedge (R \rightarrow S)$	by S2 and R3
S8.	$I \models Q \rightarrow R$	by S7 and R3
S9.	_____	_____
S10.	_____	_____
S11.	_____	_____
S12.	_____	_____
S13.	$I \models \perp$	_____

Problem 2. (20pt) DPLL

Consider the formula F

$$F : (P \vee \neg Q \vee \neg R) \wedge (\neg P \vee Q) \wedge (\neg P \vee \neg Q) \wedge (\neg Q \vee R) \wedge (\neg Q \vee \neg R).$$

Recall the DPLL algorithm that we learned in class. As explained in class, we assume that the formulas are simplified using template equivalences after replacing P with $\top$ or $\perp$ at line 7.

Algorithm 1 DPLL

Input: A PL formula F in CNF

Output: Satisfiability result (*true*: SAT, *false*: UNSAT)

```

1:  $F \leftarrow \mathbf{BCP}(F)$ 
2:  $F \leftarrow \mathbf{PLE}(F)$ 
3: if  $F = \top$  then return  $\top$ 
4: else if  $F = \perp$  then return  $\perp$ 
5: else
6:    $P \leftarrow \mathbf{ChooseVar}(F)$ 
7:   return  $\mathbf{DPLL}(F\{P \mapsto \top\})$  or  $\mathbf{DPLL}(F\{P \mapsto \perp\})$ 

```

Complete the DPLL execution on the formula F (4pt for each).

- After executing lines 1 and 2, we obtain the following intermediate formula F_1 .

$F_1 : \underline{\hspace{15em}}$

- At lines 6-7, by choosing P on the true branch, we obtain F_2 .

$F_2 : \underline{\hspace{15em}}$

By unit resolution at line 1, we deduce $\perp$.

- Now considering the other branch for P at lines 6-7, we obtain F_3 .

$F_3 : \underline{\hspace{15em}}$

After line 1, we obtain the intermediate formula F_4 .

$F_4 : \underline{\hspace{15em}}$

After line 2, we obtain $\top$ with a satisfying interpretation I (if some variables can be either $\top$ or $\perp$, write “any” for them).

$I : \{\underline{\hspace{10em}}\}$

Problem 3. (15pt) The Theory of Arrays

Determine the validity of each formula F in the theory of arrays T_A . If F is valid, simply write “valid”. If F is invalid, write “invalid” and provide a falsifying T_A interpretation. For each correct answer, you will get 5 points. For each wrong answer (including wrong interpretations), you will lose 5 points. No partial points will be given for each sub-problem.

(1) $F : a[i] = e \leftrightarrow \forall j. a\langle i \triangleleft e \rangle[j] = a[j]$

(2) $F : a[i] = p \wedge p = q \rightarrow a\langle i \triangleleft q \rangle = a$

(3) $F : a\langle i \triangleleft e \rangle[j] = e \leftrightarrow i = j \vee a[j] = e$

Problem 4. (15pt) Uninterpreted Functions

We have learned that software verification can be much more efficient by abstracting complex functions using uninterpreted function symbols. However, doing so may lead to undesired results! Provide a concrete example that demonstrates this aspect in the context of semantic equivalence checking between two programs.

Problem 5. (30pt) O/X Questions

You will get 2 points for each correct answer. You will lose 2 points for each wrong answer.

- (1) Fuzzing can be used to prove the absence of SW errors. (O, X)
- (2) Achieving 100% of branch coverage is undecidable. (O, X)
- (3) Every propositional formula has an equivalent formula in NNF. (O, X)
- (4) Every propositional formula has an equivalent formula in DNF. (O, X)
- (5) Determining the satisfiability of a DNF formula can be faster than determining the satisfiability of a CNF formula. (O, X)
- (6) Given a propositional formula F , the equisatisfiable formula F' generated by Tseitin's transformation is logically weaker than F . (O, X)
- (7) Given a satisfiable DNF formula F as input, DPLL may incorrectly determine that F is unsatisfiable. (O, X)
- (8) We cannot check the satisfiability of F using an algorithm that checks the validity, because that algorithm cannot produce a satisfying assignment even if F is satisfiable. (O, X)
- (9) We can generate partial interpretations using the clausal resolution and the unit resolution. (O, X)
- (10) A first-order theory T is decidable iff there is an algorithm that terminates with “yes” if $T \models F$ or with “no” if $T \models \neg F$. (O, X)
- (11) If a formula is valid in first-order logic, then it is also valid in every first-order theory T . (O, X)
- (12) If a formula is valid in some first-order theory T , then it is also valid in first-order logic. (O, X)
- (13) In the theory of arrays, $\forall a, v, i, j. a[i \triangleleft v][j] \neq a[j] \rightarrow i = j$. (O, X)
- (14) Implicit type casting is supported in OCaml. (O, X)
- (15) An OCaml list can contain different types of elements. (O, X)